

Blob Maintenance with Azure Functions

Student Lab: Timer Triggers, HTTP Triggers & Automated Blob Archiving

All commands use Windows Command Prompt (cmd.exe) | Fresh start — no prior lab required

Lab Overview

In this lab you will build a serverless Blob maintenance service using Azure Functions. A Timer Trigger function will automatically move files older than 7 days from an uploads container to an archive container every night at midnight. An HTTP Trigger function gives you an on-demand way to run the same logic — perfect for testing. All Azure resources are created fresh from the command line.

What You Will Build

Lab	Goal
Lab 1: Setup	Sign in to Azure CLI, set variables, create the resource group
Lab 2: Storage	Create a storage account with uploads and archive containers
Lab 3: Key Vault	Store the storage connection string securely in Key Vault
Lab 4: Function App	Scaffold a .NET 8 isolated-worker Function App project
Lab 5: Timer Trigger	Write a function that archives old blobs on a nightly cron schedule
Lab 6: HTTP Trigger	Write an on-demand function that triggers the same archive logic
Lab 7: Test Locally	Run the Function App locally and test both functions with curl
Lab 8: Deploy to Azure	Create a Function App in Azure, connect Key Vault, and deploy
Lab 9: Cleanup	Delete all resources with a single command

Prerequisites — Install These First

- Azure CLI — confirm with: `az --version`
- .NET SDK 8.0 or later — confirm with: `dotnet --version`
- Azure Functions Core Tools v4 — install once with the command below, then confirm with: `func --version`

```
npm install -g azure-functions-core-tools@4 --unsafe-perm true
```

Tip

If npm is not installed, download Node.js from <https://nodejs.org> — the LTS version includes npm. After installing Node.js, reopen cmd.exe and run the npm command above.

Important

Keep ONE cmd.exe window open for the entire lab. Variables set with the `set` command only last for the current session — if you close the window you must re-run the `set` commands from Lab 1 using your original SUFFIX value.

Lab 1: CLI Setup

Step 1 — Sign In

1. Run `az login`. A browser window will open — sign in with your Azure account.

```
az login
```

```
vaibhavgupta@FVF0995WQ85F-vaibhavgupta Day 49 July 2 Azure % az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize. Please continue the login in the web browser. If no web browser is available or if the web browser fails to open, use device code flow with 'az login --use-device-code'.

Retrieving tenants and subscriptions for the selection...
The following tenants don't contain accessible subscriptions. Use 'az login --allow-no-subscriptions' to have tenant level access.
7f7268a4-7179-4a42-9837-48255b1c21e5 'Presidio Network Solutions'

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Training-2026      e25db867-f63d-489a-8bce-7de21e5f94a1 Presidio Networked Solutions

The default is marked with an *: the default tenant is 'Presidio Networked Solutions' and subscription is 'Training-2026' (e25db867-f63d-489a-8bce-7de21e5f94a1).
Select a subscription and tenant (Type a number or Enter for no changes):
```

Step 2 — Set Variables

2. Type each line exactly. No spaces around the = sign.

```
set RG=func-lab-rg
set LOCATION=eastus
set /A SUFFIX=%RANDOM%
set SA_NAME=func-labsa%SUFFIX%
set KV_NAME=func-lab-kv-%SUFFIX%
set FUNC_NAME=func-lab-app-%SUFFIX%
set CONTAINER_UPLOADS=uploads
set CONTAINER_ARCHIVE=archive
```

3. Confirm your values printed correctly:

```
echo %SA_NAME%
echo %KV_NAME%
echo %FUNC_NAME%
```

```
vaibhavgupta@FVF0995WQ85F-vaibhavgupta Day 49 July 2 Azure % export RG=func-lab-rg
export LOCATION=eastus
export SUFFIX=$RANDOM
export SA_NAME=func-labsa$SUFFIX
export KV_NAME=func-lab-kv-$SUFFIX
export FUNC_NAME=func-lab-app-$SUFFIX
export CONTAINER_UPLOADS=uploads
export CONTAINER_ARCHIVE=archive
vaibhavgupta@FVF0995WQ85F-vaibhavgupta Day 49 July 2 Azure % echo %SA_NAME%
echo $KV_NAME
echo $FUNC_NAME
echo $SUFFIX
func-labsa11215
func-lab-kv-11215
func-lab-app-11215
11215
vaibhavgupta@FVF0995WQ85F-vaibhavgupta Day 49 July 2 Azure %
```

Write your values here — you will need them if you ever need to restore the variables:

SA_NAME value

funclabsa11215

KV_NAME value

func-lab-kv-11215

FUNC_NAME value

func-lab-app-11215

SUFFIX value

11215

Step 3 — Create the Resource Group

```
az group create --name %RG% --location %LOCATION%
```

```
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure % az group create --name $RG --location $LOCATION
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/func-lab-rg",
  "location": "eastus",
  "managedBy": null,
  "name": "func-lab-rg",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure %
```

Lab 2: Storage Account

The function will move blobs between two containers: uploads (where files land) and archive (where old files go). Both containers live in the same storage account.

Step 1 — Create the Storage Account

```
az storage account create ^  
  --name %SA_NAME% ^  
  --resource-group %RG% ^  
  --location %LOCATION% ^  
  --sku Standard_LRS ^  
  --kind StorageV2
```

```
{  
  "immutableStorageWithVersioning": null,  
  "isHnsEnabled": null,  
  "isLocalUserEnabled": null,  
  "isSftpEnabled": null,  
  "isSkuConversionBlocked": null,  
  "keyCreationTime": {  
    "key1": "2026-07-02T08:45:29.685887+00:00",  
    "key2": "2026-07-02T08:45:29.685887+00:00"  
  },  
  "keyPolicy": null,  
  "kind": "StorageV2",  
  "largeFileSharesState": null,  
  "lastGeoFailoverTime": null,  
  "location": "eastus",  
  "minimumTlsVersion": "Tls1_0",  
  "name": "func1absa11215",  
  "networkRuleSet": {  
    "bypass": "None",  
    "defaultAction": "Allow",  
    "ipRules": [],  
    "ipRules": [],  
    "resourceAccessRules": null,  
    "virtualNetworkRules": []  
  },  
  "placement": null,  
  "primaryEndpoints": {  
    "blob": "https://func1absa11215.blob.core.windows.net/",  
    "dfs": "https://func1absa11215.dfs.core.windows.net/",  
    "file": "https://func1absa11215.file.core.windows.net/",  
    "internetEndpoints": null,  
    "ipV6Endpoints": null,  
    "microsoftEndpoints": null,  
    "queue": "https://func1absa11215.queue.core.windows.net/",  
    "table": "https://func1absa11215.table.core.windows.net/",  
    "web": "https://func1absa11215.z13.web.core.windows.net/"  
  },  
  "primaryLocation": "eastus",  
  "privateEndpointConnections": [],  
  "provisioningState": "Succeeded",  
  "publicNetworkAccess": null,  
  "resourceGroup": "func-lab-rg",  
  "routingPreference": null,  
  "sasPolicy": null,  
  "secondaryEndpoints": null,  
  "secondaryLocation": null,  
  "sku": {  
    "name": "Standard_LRS",  
    "tier": "Standard"  
  },  
  "statusOfPrimary": "available",  
  "statusOfSecondary": null,  
  "storageAccountSkuConversionStatus": null,  
  "systemData": null,  
  "tags": {},  
  "type": "Microsoft.Storage/storageAccounts",  
  "zones": null  
}
```

vaibhavgupta@VF0908WQ05F-vaibhavgupta Day 49 July 2 Azure % az storage container create \

Step 2 — Create the Two Containers

```
az storage container create ^  
  --name %CONTAINER_UPLOADS% ^  
  --account-name %SA_NAME% ^  
  --auth-mode login
```

```
az storage container create ^
--name %CONTAINER_ARCHIVE% ^
--account-name %SA_NAME% ^
--auth-mode login
```

```
ncps177f0e1b3a11215.blob.core.windows.net/
vaibhavgupta@FVF0905WQ05F--vaibhavgupta Day 49 July 2 Azure % az storage container create \
--name $CONTAINER_UPLOADS \
--account-name $SA_NAME \
--auth-mode login
{
  "created": true
}
vaibhavgupta@FVF0905WQ05F--vaibhavgupta Day 49 July 2 Azure % az storage container create \
--name $CONTAINER_ARCHIVE \
--account-name $SA_NAME \
--auth-mode login
{
  "created": true
}
vaibhavgupta@FVF0905WQ05F--vaibhavgupta Day 49 July 2 Azure % █
```

Step 3 — Capture the Connection String

```
for /f "delims=" %a in ('az storage account show-connection-string --name %SA_NAME% --
resource-group %RG% --query connectionString -o tsv') do set CONN_STRING=%a
```

Tip

Verify it was captured: `echo %CONN_STRING%` — it should start with `DefaultEndpointsProtocol=https`

```
vaibhavgupta@FVF0905WQ05F--vaibhavgupta Day 49 July 2 Azure % export CONN_STRING=$(az storage account show-connection-string \
--name $SA_NAME \
--resource-group $RG \
--query connectionString \
-o tsv)
vaibhavgupta@FVF0905WQ05F--vaibhavgupta Day 49 July 2 Azure % echo $CONN_STRING
DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=func1absa11215;AccountKey=TJ9DpNmbYK1QDJ+TVH/OFDffS1fE7ziiC16jGKV2Jg/vwL4NikZ5U0kLQu2RfHP3cjFDli1+zOwj+ASt0ejsqw==;BlobEndpoint=https://func1absa11215.blob.core.windows.net;/FileEndpoint=https://func1absa11215.file.core.windows.net;/QueueEndpoint=https://func1absa11215.queue.core.windows.net;/TableEndpoint=https://func1absa11215.table.core.windows.net/
vaibhavgupta@FVF0905WQ05F--vaibhavgupta Day 49 July 2 Azure % █
```

DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=func1absa11215;AccountKey=TJ9DpNmbYK1QDJ+TVH/OFDffS1fE7ziiC16jGKV2Jg/vwL4NikZ5U0kLQu2RfHP3cjFDli1+zOwj+ASt0ejsqw==;BlobEndpoint=https://func1absa11215.blob.core.windows.net;/FileEndpoint=https://func1absa11215.file.core.windows.net;/QueueEndpoint=https://func1absa11215.queue.core.windows.net;/TableEndpoint=https://func1absa11215.table.core.windows.net/

Lab 3: Key Vault

The connection string will be stored in Key Vault. In Azure, the Function App will read it via a Key Vault Reference — a special app setting syntax that tells Azure to fetch the value from Key Vault automatically, with no code changes needed.

Step 1 — Create the Vault

```
az keyvault create ^  
  --name %KV_NAME% ^  
  --resource-group %RG% ^  
  --location %LOCATION%
```

```
--location $LOCATION  
{  
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/func-lab-rg/providers/Microsoft.KeyVault/vaults/func-lab-kv-11215",  
  "location": "eastus",  
  "name": "func-lab-kv-11215",  
  "properties": {  
    "accessPolicies": [],  
    "createMode": null,  
    "enablePurgeProtection": null,  
    "enableRbacAuthorization": true,  
    "enableSoftDelete": true,  
    "enabledForDeployment": false,  
    "enabledForDiskEncryption": false,  
    "enabledForTemplateDeployment": false,  
    "hsmPoolResourceId": null,  
    "networkAcls": null,  
    "privateEndpointConnections": null,  
    "provisioningState": "Succeeded",  
    "publicNetworkAccess": "Enabled",  
    "sku": {  
      "family": "A",  
      "name": "standard"  
    },  
    "softDeleteRetentionInDays": 90,  
    "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d8c",  
    "vaultUri": "https://func-lab-kv-11215.vault.azure.net/"  
  },  
  "resourceGroup": "func-lab-rg",  
  "systemData": {  
    "createdAt": "2026-07-02T09:02:29.478000+00:00",  
    "createdBy": "vgupta1@presidio.com",  
    "createdByType": "User",  
    "lastModifiedAt": "2026-07-02T09:02:29.478000+00:00",  
    "lastModifiedBy": "vgupta1@presidio.com",  
    "lastModifiedByType": "User"  
  },  
  "tags": {},  
  "type": "Microsoft.KeyVault/vaults"  
}  
vaibhavgupta@FVF0905WQ05F-vaibhavgupta Day 49 July 2 Azure %
```

Step 2 — Grant Yourself Access

4. Capture your Azure AD object ID and set an access policy:

```
for /f "delims=" %a in ('az ad signed-in-user show --query id -o tsv') do set MY_OID=%a
```

```
az keyvault set-policy ^
--name %KV_NAME% ^
--object-id %MY_OID% ^
--secret-permissions get set list delete
```

```
{
  "type": "Microsoft.KeyVault/vaults"
}
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure % export MY_OID=$(az ad signed-in-user show --query id -o tsv)
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure % echo $MY_OID
18f12757-ddc1-413f-88db-4ed9843b2309
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure % █
```

Step 3 — Store the Connection String

```
az keyvault secret set ^
--vault-name %KV_NAME% ^
--name "BlobStorageConnectionString" ^
--value "%CONN_STRING%"
```

```
{
  "softDeleteRetentionInDays": 90,
  "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d8c",
  "vaultUri": "https://func-lab-kv-11215.vault.azure.net/"
},
{
  "resourceGroup": "func-lab-rg",
  "systemData": {
    "createdAt": "2026-07-02T09:24:46.151000+00:00",
    "createdBy": "vgupta1@presidio.com",
    "createdByType": "User",
    "lastModifiedAt": "2026-07-02T09:27:39.433000+00:00",
    "lastModifiedBy": "vgupta1@presidio.com",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.KeyVault/vaults"
}
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure % echo $CONN_STRING
DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=funcbsa11215;AccountKey=TJ9DpNmbYK1QDJ+TVH/OFDffS1fE7ziiC16jGKV2Jg/vwL4NikZ5U0kLQu2RfHP3cjFDIi1+zOwj+ASt0ejsqw==;BlobEndpoint=https://funcbsa11215.blob.core.windows.net;/FileEndpoint=https://funcbsa11215.file.core.windows.net;/QueueEndpoint=https://funcbsa11215.queue.core.windows.net;/TableEndpoint=https://funcbsa11215.table.core.windows.net/
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 49 July 2 Azure % █
```

Important

This is the last time the connection string is handled directly. From this point the secret lives only inside Key Vault.

DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=funcbsa11215;AccountKey=TJ9DpNmbYK1QDJ+TVH/OFDffS1fE7ziiC16jGKV2Jg/vwL4NikZ5U0kLQu2RfHP3cjFDIi1+zOwj+ASt0ejsqw==;BlobEndpoint=https://funcbsa11215.blob.core.windows.net;/FileEndpoint=https://funcbsa11215.file.core.windows.net;/QueueEndpoint=https://funcbsa11215.queue.core.windows.net;/TableEndpoint=https://funcbsa11215.table.core.windows.net/

Endpoint=https://funclabsa11215.queue.core.windows.net/;TableEndpoint=https://funclabsa11215.table.core.windows.net/

Lab 4: Scaffold the Function App

Step 1 — Create the Project

5. Choose a folder on your machine to work in, then cd into it.
6. Run the following to create the Function App project:

```
func init BlobArchiveFunctions --worker-runtime dotnet-isolated --target-framework net8.0
cd BlobArchiveFunctions
```

Tip

func init creates a .csproj file, a Program.cs, a host.json, and a local.settings.json. The dotnet-isolated model is the modern .NET approach for Azure Functions.

```
vaibhav@vaibhav:~$ func init BlobArchiveFunctions --worker-runtime dotnet-isolated --target-framework net8.0
vaibhav@vaibhav:~$ cd BlobArchiveFunctions
vaibhav@vaibhav:~/BlobArchiveFunctions$ find . -maxdepth 2
.
./BlobArchiveFunctions
./BlobArchiveFunctions/obj
./BlobArchiveFunctions/host.json
./BlobArchiveFunctions/BlobArchiveFunctions.csproj
./BlobArchiveFunctions/.gitignore
./BlobArchiveFunctions/Properties
./BlobArchiveFunctions/.vscode
./BlobArchiveFunctions/local.settings.json
./BlobArchiveFunctions/Program.cs
vaibhav@vaibhav:~/BlobArchiveFunctions$ dotnet add package Azure.Storage.Blobs
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.203/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.203/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Azure.Storage.Blobs' into project '/Users/vaibhav/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : GET https://api.nuget.org/v3/registrations-gr-semver2/azure.storage.blobs/index.json 449ms
info : OK https://api.nuget.org/v3/registrations-gr-semver2/azure.storage.blobs/index.json 449ms
info : Restoring packages for /Users/vaibhav/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj...
info : CACHE https://api.nuget.org/v3-flatcontainer/system.diagnostics.diagnosticsource/index.json
info : GET https://api.nuget.org/v3-flatcontainer/system.valuetuple/index.json
info : GET https://api.nuget.org/v3-flatcontainer/system.diagnostics.diagnosticsource/6.0.1/system.diagnostics.diagnosticsource.6.0.1.nupkg
info : OK https://api.nuget.org/v3-flatcontainer/system.diagnostics.diagnosticsource/6.0.1/system.diagnostics.diagnosticsource.6.0.1.nupkg 82ms
info : OK https://api.nuget.org/v3-flatcontainer/system.valuetuple/index.json 359ms
info : GET https://api.nuget.org/v3-flatcontainer/system.valuetuple/4.5.0/system.valuetuple.4.5.0.nupkg
info : OK https://api.nuget.org/v3-flatcontainer/system.valuetuple/4.5.0/system.valuetuple.4.5.0.nupkg 24ms
info : Installed System.ValueTuple 4.5.0 from https://api.nuget.org/v3/index.json to /Users/vaibhav/.nuget/packages/system.valuetuple/4.5.0 with content hash okurQJ06NRE/apDIP23aj30hpinM3+f08wO1B/c
SqtLOuEupf65-96s1022w4edrfvovP/PIAIBUJW4RQ==.
info : Installed System.Diagnostics.DiagnosticsSource 6.0.1 from https://api.nuget.org/v3/index.json to /Users/vaibhav/.nuget/packages/system.diagnostics.diagnosticsource/6.0.1 with content hash Kil
YD2k2J820983pWu0qCkFj8R8Wqjd0KwW19WzYD0X3cmFX0vqTQ1L14W19EuF3rl.0+MCLTbqMq==.
info : CACHE https://api.nuget.org/v3-vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/2026.07.02.06.04.35/vulnerability.update.json
info : Package 'Azure.Storage.Blobs' is compatible with all the specified frameworks in project '/Users/vaibhav/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : PackageReference for package 'Azure.Storage.Blobs' version '12.29.1' added to file '/Users/vaibhav/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : Generating MSBuild file /Users/vaibhav/AzureLabs/BlobArchiveFunctions/obj/BlobArchiveFunctions.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: /Users/vaibhav/AzureLabs/BlobArchiveFunctions/obj/project.assets.json
log : Restored /Users/vaibhav/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj (in 983 ms).
vaibhav@vaibhav:~/BlobArchiveFunctions$ dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Timer
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.203/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.203/trustedroots/timestampctl.pem'.
```

Step 2 — Add the Blob Storage Package

```
dotnet add package Azure.Storage.Blobs
```

```
Info : Writing assets file to disk. Path: /Users/vaibhavgupta/AzureLabs/BlobArchiveFunctions/obj/project.assets.json
log : Restored /Users/vaibhavgupta/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj (in 983 ms).
vaibhavgupta@FVF998WQ85F~vaibhavgupta BlobArchiveFunctions % dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Timer
Info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.283/trustedroots/codesignctl.pem'.
Info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.283/trustedroots/timestampctl.pem'.
Info : Adding PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' into project '/Users/vaibhavgupta/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
Info : GET https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.azure.functions.worker.extensions.timer/index.json
Info : OK https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.azure.functions.worker.extensions.timer/index.json 390ms
Info : Restoring packages for /Users/vaibhavgupta/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj...
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/index.json
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/index.json 1874ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/4.3.1/microsoft.azure.functions.worker.extensions.timer.4.3.1.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/4.3.1/microsoft.azure.functions.worker.extensions.timer.4.3.1.nupkg 23ms
Info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/1.18.0/microsoft.azure.functions.worker.core.1.18.0.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/1.18.0/microsoft.azure.functions.worker.core.1.18.0.nupkg 23ms
Info : Installed Microsoft.Azure.Functions.Worker.Core 1.18.0 from https://api.nuget.org/v3/index.json to /Users/vaibhavgupta/.nuget/packages/microsoft.azure.functions.worker.core/1.18.0 with content hash AFHJMjPB73hmmWGFbrosGMRaSertGrG6j9DcCzpt03ymjtM298asD9zLekAniRj2hQBMF0SsTmlaq/cKdVQ0==.
Info : Installed Microsoft.Azure.Functions.Worker.Extensions.Timer 4.3.1 from https://api.nuget.org/v3/index.json to /Users/vaibhavgupta/.nuget/packages/microsoft.azure.functions.worker.extensions.timer/4.3.1 with content hash UNKWSuel1J2SG2e60C+s713y8kXZCKYnF/OcssXIONY+vm+AP+zm/wPR7i35xeDgOp5oCKHeu5YshbZsfCrg==.
Info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
Info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/vulnerability.base.json
```

Step 3 — Add the Timer Extension

```
dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Timer
```

Step 4 — Create the Function Files

7. Add the Timer Trigger function file:

```
func new --name NightlyArchive --template "Timer trigger" --language C#
```

8. Add the HTTP Trigger function file:

```
func new --name ManualArchive --template "HTTP trigger" --language C#
```

```
log : Restored /Users/vaibhavgupta/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj (in 1.47 sec).
vaibhavgupta@FVF998WQ85F~vaibhavgupta BlobArchiveFunctions % func new --name NightlyArchive --template "Timer trigger"
Template: Timer trigger
Function name: [Timer trigger] NightlyArchive
The function "NightlyArchive" was created successfully from the "Timer trigger" template.
vaibhavgupta@FVF998WQ85F~vaibhavgupta BlobArchiveFunctions % func new --name ManualArchive --template "HTTP trigger"
Template: HTTP trigger
Function name: [HTTP trigger] ManualArchive
The function "ManualArchive" was created successfully from the "HTTP trigger" template.
vaibhavgupta@FVF998WQ85F~vaibhavgupta BlobArchiveFunctions % ls
BlobArchiveFunctions.csproj  local.settings.json  NightlyArchive.cs  Program.cs  readme.md
host.json  ManualArchive.cs  obj  Properties
vaibhavgupta@FVF998WQ85F~vaibhavgupta BlobArchiveFunctions % dotnet build
Restore complete (0.4s)
WorkerExtensions net8.0 succeeded (5.8s) -> obj/Debug/net8.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
BlobArchiveFunctions net8.0 succeeded (5.0s) -> bin/Debug/net8.0/BlobArchiveFunctions.dll
Build succeeded in 26.1s
vaibhavgupta@FVF998WQ85F~vaibhavgupta BlobArchiveFunctions %
```

Lab 5: Write the Timer Trigger Function

The Timer Trigger function runs automatically on a cron schedule. You will replace the generated placeholder code with logic that scans the uploads container and moves files older than 7 days to archive.

Step 1 — Create the Archive Service

9. Create a new file called `BlobArchiveService.cs` in the `BlobArchiveFunctions` folder with the following content:

```
using Azure.Storage.Blobs;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class BlobArchiveService
{
    private readonly ILogger<BlobArchiveService> _logger;
    private readonly string _connectionString;

    public BlobArchiveService(ILogger<BlobArchiveService> logger, IConfiguration config)
    {
        _logger = logger;
        _connectionString = config["BlobStorageConnectionString"]
            ?? throw new InvalidOperationException("BlobStorageConnectionString is not configured.");
    }

    public async Task<int> ArchiveOldBlobsAsync(int olderThanDays = 7)
    {
        var serviceClient = new BlobServiceClient(_connectionString);
        var uploads = serviceClient.GetBlobContainerClient("uploads");
        var archive = serviceClient.GetBlobContainerClient("archive");

        await archive.CreateIfNotExistsAsync();

        int count = 0;
        var cutoff = DateTimeOffset.UtcNow.AddDays(-olderThanDays);

        await foreach (var blobItem in uploads.GetBlobsAsync())
        {
            if (blobItem.Properties.LastModified < cutoff)
            {
                var sourceBlob = uploads.GetBlobClient(blobItem.Name);
                var destBlob = archive.GetBlobClient(blobItem.Name);

                await destBlob.StartCopyFromUriAsync(sourceBlob.Uri);
                await sourceBlob.DeleteAsync();

                _logger.LogInformation("Archived: {BlobName}", blobItem.Name);
                count++;
            }
        }
    }
}
```

```
        }  
    }  
  
    _logger.LogInformation("Archive run complete. Files moved: {Count}", count);  
    return count;  
}  
}
```

Step 2 — Register the Service in Program.cs

10. Open Program.cs and replace its contents with:

```
using Microsoft.Azure.Functions.Worker;  
using Microsoft.Extensions.DependencyInjection;  
using Microsoft.Extensions.Hosting;  
using BlobArchiveFunctions;  
  
var host = new HostBuilder()  
    .ConfigureFunctionsWorkerDefaults()  
    .ConfigureServices(services =>  
    {  
        services.AddTransient<BlobArchiveService>();  
    })  
    .Build();  
  
host.Run();
```

Step 3 — Replace the NightlyArchive Function

11. Open NightlyArchive.cs and replace its contents with:

```
using Microsoft.Azure.Functions.Worker;  
using Microsoft.Extensions.Logging;  
  
namespace BlobArchiveFunctions;  
  
public class NightlyArchive  
{  
    private readonly BlobArchiveService _archiveService;  
    private readonly ILogger<NightlyArchive> _logger;  
  
    public NightlyArchive(BlobArchiveService archiveService, ILogger<NightlyArchive>  
logger)  
    {  
        _archiveService = archiveService;  
        _logger = logger;  
    }  
  
    // Runs every day at midnight UTC  
    // Cron format: {second} {minute} {hour} {day} {month} {day-of-week}  
    [Function("NightlyArchive")]  
    public async Task Run([TimerTrigger("0 0 0 * * *")] TimerInfo myTimer)
```

```
{
    _logger.LogInformation("NightlyArchive triggered at: {Time}", DateTime.UtcNow);
    await _archiveService.ArchiveOldBlobsAsync(olderThanDays: 7);
}
```

Tip

The cron expression `0 0 0 * * *` means: at second 0, minute 0, hour 0, every day, every month, every day of the week — i.e. midnight UTC daily.

Lab 6: Write the HTTP Trigger Function

The HTTP Trigger lets you call the same archive logic on demand — useful for testing without waiting for midnight. It also accepts an optional query parameter so you can test with a shorter age window.

Step 1 — Replace the ManualArchive Function

12. Open ManualArchive.cs and replace its contents with:

```
using System.Net;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class ManualArchive
{
    private readonly BlobArchiveService _archiveService;
    private readonly ILogger<ManualArchive> _logger;

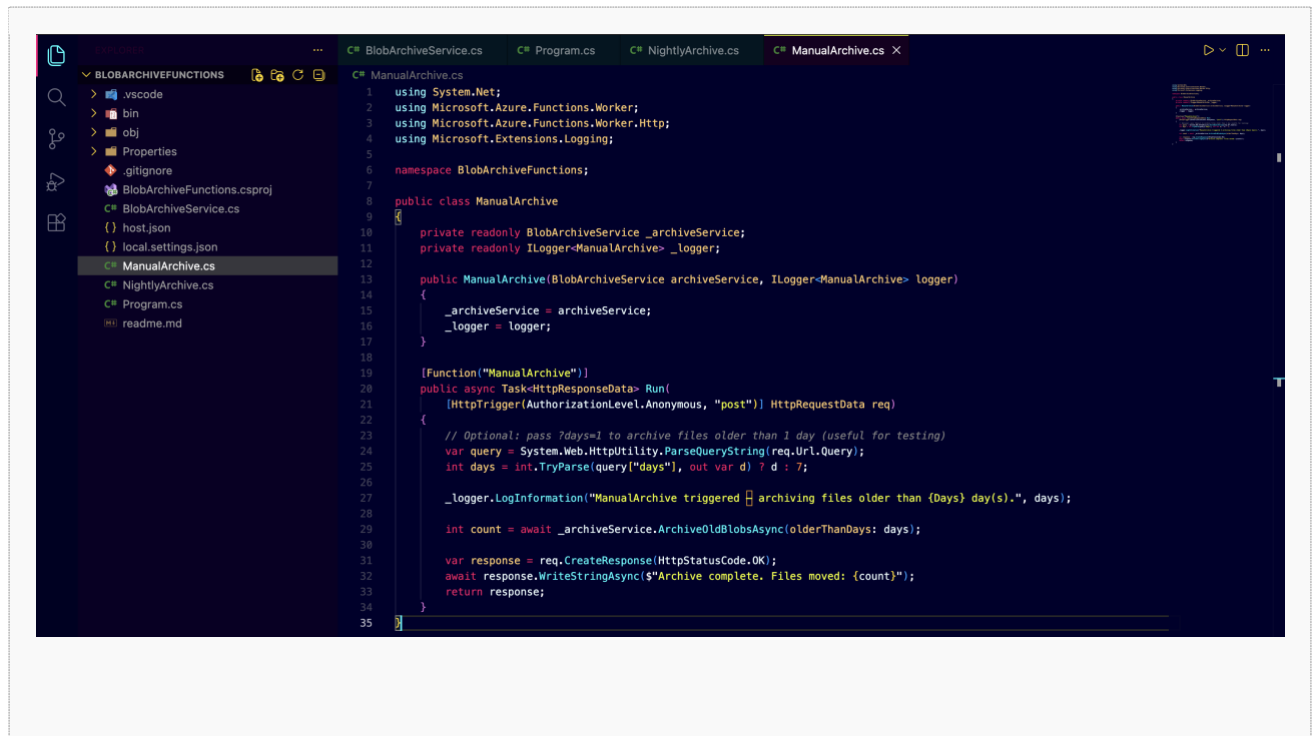
    public ManualArchive(BlobArchiveService archiveService, ILogger<ManualArchive> logger)
    {
        _archiveService = archiveService;
        _logger = logger;
    }

    [Function("ManualArchive")]
    public async Task<HttpResponseBody> Run(
        [HttpTrigger(AuthorizationLevel.Anonymous, "post")] HttpRequestData req)
    {
        // Optional: pass ?days=1 to archive files older than 1 day (useful for testing)
        var query = System.Web.HttpUtility.ParseQueryString(req.Url.Query);
        int days = int.TryParse(query["days"], out var d) ? d : 7;

        _logger.LogInformation("ManualArchive triggered – archiving files older than {Days} day(s).", days);

        int count = await _archiveService.ArchiveOldBlobsAsync(olderThanDays: days);

        var response = req.CreateResponse(HttpStatusCode.OK);
        await response.WriteStringAsync($"Archive complete. Files moved: {count}");
        return response;
    }
}
```



The screenshot shows the Visual Studio Code editor with a C# project for Azure Functions. The file explorer on the left shows the project structure, including the `ManualArchive.cs` file. The main editor displays the code for `ManualArchive.cs`, which implements a function triggered by an HTTP request. The code uses `BlobArchiveService` to archive files older than a specified number of days.

```
1 using System.Net;
2 using Microsoft.Azure.Functions.Worker;
3 using Microsoft.Azure.Functions.Worker.Http;
4 using Microsoft.Extensions.Logging;
5
6 namespace BlobArchiveFunctions;
7
8 public class ManualArchive
9 {
10     private readonly BlobArchiveService _archiveService;
11     private readonly ILogger<ManualArchive> _logger;
12
13     public ManualArchive(BlobArchiveService archiveService, ILogger<ManualArchive> logger)
14     {
15         _archiveService = archiveService;
16         _logger = logger;
17     }
18
19     [Function("ManualArchive")]
20     public async Task<HttpStatusCode> Run(
21         [HttpTrigger(AuthorizationLevel.Anonymous, "post")] HttpRequestData req)
22     {
23         // Optional: pass ?days=1 to archive files older than 1 day (useful for testing)
24         var query = System.Web.HttpUtility.ParseQueryString(req.Url.Query);
25         int days = int.TryParse(query["days"], out var d) ? d : 7;
26
27         _logger.LogInformation("ManualArchive triggered {d} archiving files older than {Days} day(s).", days);
28
29         int count = await _archiveService.ArchiveOldBlobsAsync(olderThanDays: days);
30
31         var response = req.CreateResponse(HttpStatusCode.OK);
32         await response.WriteStringAsync($"Archive complete. Files moved: {count}");
33         return response;
34     }
35 }
```

Lab 7: Test Locally

Before deploying to Azure, test the functions on your machine. The local settings file acts as a stand-in for Azure app settings when running locally.

Step 1 — Configure local.settings.json

13. Open local.settings.json (in the BlobArchiveFunctions folder) and replace its contents with your actual connection string:

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated",
    "BlobStorageConnectionString": "PASTE-YOUR-CONNECTION-STRING-HERE"
  }
}
```

Important

Replace PASTE-YOUR-CONNECTION-STRING-HERE with the value from echo %CONN_STRING% in your cmd.exe session. local.settings.json is already in .gitignore by default — never commit it to source control.

Step 2 — Build the Project

```
dotnet build
```

```
vaibhavgupta@VF0905WQ05F-vaibhavgupta BlobArchiveFunctions % dotnet build
func start
Restore complete (0.3s)
  WorkerExtensions net8.0 succeeded (1.3s) → obj/Debug/net10.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
  BlobArchiveFunctions net10.0 succeeded (1.7s) → bin/Debug/net10.0/BlobArchiveFunctions.dll

Build succeeded in 3.8s
Running 'func start' directly against a .NET Isolated project may not correctly load function extensions. Use 'dotnet run' instead, which builds the project and starts the Functions host from the correct output directory.
Determining projects to restore...
All projects are up-to-date for restore.
Determining projects to restore...
All projects are up-to-date for restore.
WorkerExtensions -> /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/obj/Debug/net10.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
BlobArchiveFunctions -> /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/bin/output/BlobArchiveFunctions.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.15

Azure Functions Core Tools
Core Tools Version: 4.12.1-7f573ec7a43edc895383eb81b78fe86ead2b787b (64-bit)
Function Runtime Version: 4.1048.208.26158

[2026-07-03T06:41:20.119Z] Found /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/BlobArchiveFunctions.csproj. Using for user secrets file configuration.
[2026-07-03T06:41:21.287Z] Worker process started and initialized.

Functions:

    ManualArchive: {POST} http://localhost:7071/api/ManualArchive
    NightlyArchive: timerTrigger

For detailed output, run func with --verbose flag.
[2026-07-03T06:41:27.618Z] Host lock lease acquired by instance ID '00000000000000000000000432CF552'.
```


Step 3 — Start the Function App

```
func start
```

The Functions runtime will start and print the URLs for both functions. Look for two lines like:

```
NightlyArchive: timerTrigger
ManualArchive: [POST] http://localhost:7071/api/ManualArchive
```

```
vaibhav@vaibhav:~/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions$ dotnet build
Restore complete (0.3s)
  WorkerExtensions net8.0 succeeded (1.3s) → obj/Debug/net10.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
  BlobArchiveFunctions net10.0 succeeded (1.7s) → bin/Debug/net10.0/BlobArchiveFunctions.dll

Build succeeded in 3.8s
Running 'func start' directly against a .NET Isolated project may not correctly load function extensions. Use 'dotnet run' instead, which builds the project and starts the Functions host from the correct output directory.
  Determining projects to restore...
  All projects are up-to-date for restore.
  Determining projects to restore...
  All projects are up-to-date for restore.
  WorkerExtensions -> /Users/vaibhav@vaibhav/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/obj/Debug/net10.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
  BlobArchiveFunctions -> /Users/vaibhav@vaibhav/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/bin/output/BlobArchiveFunctions.dll

Build succeeded.
  0 Warning(s)
  0 Error(s)

Time Elapsed 00:00:02.15

Azure Functions Core Tools
Core Tools Version: 4.12.1+7f573ec7a43edc895383eb61b78fa6ead2b787b (64-bit)
Function Runtime Version: 4.1048.200.26150

[2026-07-03T06:14:12.011Z] Found /Users/vaibhav@vaibhav/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/BlobArchiveFunctions.csproj. Using for user secrets file configuration.
[2026-07-03T06:14:12.129Z] Worker process started and initialized.

Functions:

    ManualArchive: [POST] http://localhost:7071/api/ManualArchive
    NightlyArchive: timerTrigger

For detailed output, run func with --verbose flag.
[2026-07-03T06:14:12.129Z] Host lock lease acquired by instance ID '00000000000000000000000000000000432CF552'.
```

Step 4 — Upload a Test File to Blob Storage

14. Open a SECOND cmd.exe window (leave func start running in the first). Upload a test file:

```
az storage blob upload ^
--account-name %SA_NAME% ^
--container-name uploads ^
--name testfile.txt ^
--data "hello from the lab" ^
--auth-mode login
```

Tip

For testing the archive logic without waiting 7 days, you will use the ?days=0 query parameter in the next step — this archives ALL files regardless of age.

Step 5 — Trigger ManualArchive

15. In the second cmd.exe window, call the HTTP trigger with days=0 to archive everything immediately:

```
curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
```

Expected Result

Expected response: Archive complete. Files moved: 1

```
vaibhavgupta@FVF995WQ65F-vaibhavgupta BlobArchiveFunctions % az storage blob upload \
  --account-name func1absa13938 \
  --container-name uploads \
  --name testfile.txt \
  --data "hello from the lab" \
  --auth-mode key
Argument '--data' is in preview and under development. Reference and support levels: https://aka.ms/CLI_refstatus

There are no credentials provided in your command and environment, we will query for account key for your storage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credentials.

You also can add '--auth-mode login' in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information about environment variable usage.
Finished[#####] 100.0000%
{
  "client_request_id": "51a29690-76ad-11f1-9784-02de72963a13",
  "content_md5": "xth8BgnW/kbAfeF3PNJXQ==",
  "date": "2026-07-03T07:03:40+00:00",
  "encryption_key_sha256": null,
  "encryption_scope": null,
  "etag": "\"0x80ED80136616884\"",
  "lastModified": "2026-07-03T07:03:40+00:00",
  "request_id": "db514119-101e-0049-66ba-0a97fa600000",
  "request_server_encrypted": true,
  "structured_body": null,
  "version": "2026-04-06",
  "version_id": null
}
vaibhavgupta@FVF995WQ65F-vaibhavgupta BlobArchiveFunctions % curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
Archive complete. Files moved: 1
vaibhavgupta@FVF995WQ65F-vaibhavgupta BlobArchiveFunctions % az storage blob list --account-name func1absa13938 --container-name archive --auth-mode key -o table

There are no credentials provided in your command and environment, we will query for account key for your storage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credentials.

You also can add '--auth-mode login' in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information about environment variable usage.
Name          Blob Type  Blob Tier  Length  Content Type  Last Modified  Snapshot
-----
testfile.txt  BlobBlob  Hot        18      application/octet-stream  2026-07-03T07:03:49+00:00
vaibhavgupta@FVF995WQ65F-vaibhavgupta BlobArchiveFunctions %
```

Step 6 — Verify the Blob Moved

16. Confirm testfile.txt is now in the archive container and gone from uploads:

```
az storage blob list --account-name %SA_NAME% --container-name archive --auth-mode login -o table
az storage blob list --account-name %SA_NAME% --container-name uploads --auth-mode login -o table
```

```
vaibhavgupta@FVF995WQ65F-vaibhavgupta BlobArchiveFunctions % az storage blob list --account-name func1absa13938 --container-name archive --auth-mode key -o table

There are no credentials provided in your command and environment, we will query for account key for your storage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credentials.

You also can add '--auth-mode login' in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information about environment variable usage.
Name          Blob Type  Blob Tier  Length  Content Type  Last Modified  Snapshot
-----
testfile.txt  BlobBlob  Hot        18      application/octet-stream  2026-07-03T07:03:49+00:00
vaibhavgupta@FVF995WQ65F-vaibhavgupta BlobArchiveFunctions %
```

```
}
vaibhavgupta@FVF0905WQ85F-vaibhavgupta BlobArchiveFunctions % curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
Archive complete. Files moved: 1
vaibhavgupta@FVF0905WQ85F-vaibhavgupta BlobArchiveFunctions % az storage blob list --account-name funclabsai3938 --container-name archive --auth-mode key -o table

There are no credentials provided in your command and environment, we will query for account key for your storage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credentials.

You also can add '--auth-mode login' in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information about environment variable usage.

```

Name	Blob Type	Blob Tier	Length	Content Type	Last Modified	Snapshot
testfile.txt	BlockBlob	Hot	18	application/octet-stream	2026-07-03T07:03:49+00:00	

```
vaibhavgupta@FVF0905WQ85F-vaibhavgupta BlobArchiveFunctions % az storage blob list --account-name funclabsai3938 --container-name uploads --auth-mode key -o table

There are no credentials provided in your command and environment, we will query for account key for your storage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credentials.

You also can add '--auth-mode login' in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information about environment variable usage.
vaibhavgupta@FVF0905WQ85F-vaibhavgupta BlobArchiveFunctions %
```

Lab 8: Deploy to Azure

Now deploy the Function App to Azure. The deployed version will read the connection string from Key Vault via a Key Vault Reference — a special app setting syntax that tells Azure to fetch the value automatically. Your function code stays exactly the same.

Step 1 — Create the Function App in Azure

17. Stop func start in the first window (Ctrl+C), then run:

```
az functionapp create ^
--name %FUNC_NAME% ^
--resource-group %RG% ^
--storage-account %SA_NAME% ^
--consumption-plan-location %LOCATION% ^
--runtime dotnet-isolated ^
--functions-version 4 ^
--os-type Windows
```

```
"clientCertEnabled": false,
"clientCertExclusionPaths": null,
"clientCertMode": "Required",
"cloningInfo": null,
"containerSize": 1036,
"customDomainVerificationId": "46807E15E4B3288A1BC091EA4477FF9CF6E3CE676FACDEA8663A4E096E2CCC001",
"dailyMemoryTimeQuota": 0,
"daprConfig": null,
"defaultHostName": "func-lab-app-13938.azurewebsites.net",
"dnsConfiguration": {
  "dnsAltServer": null,
  "dnsLegacySortOrder": null,
  "dnsMaxCacheTimeout": null,
  "dnsRetryAttemptCount": null,
  "dnsRetryAttemptTimeout": null,
  "dnsServers": null
},
"enabled": true,
"enabledHostNames": [
  "func-lab-app-13938.azurewebsites.net",
  "func-lab-app-13938.scm.azurewebsites.net"
],
"endToEndEncryptionEnabled": false,
"extendedLocation": null,
"functionAppConfig": null,
"hostNameSslStates": [
  {
    "hostType": "Standard",
    "name": "func-lab-app-13938.azurewebsites.net",
    "sslState": "Disabled",
    "thumbprint": null,
    "toUpdate": null,
    "virtualIp": null
  },
  {
    "hostType": "Repository",
    "name": "func-lab-app-13938.scm.azurewebsites.net",
    "sslState": "Disabled",
    "thumbprint": null,
    "toUpdate": null,
    "virtualIp": null
  }
],
"hostNames": [
  "func-lab-app-13938.azurewebsites.net"
],
"hostNamesDisabled": false,
"hostingEnvironmentProfile": null,
"httpsOnly": false,
"hyperV": false,
"id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/func-lab-rg/providers/Microsoft.Web/sites/func-lab-app-13938",
"identity": null,
"inProgressOperationId": null,
"ipMode": "IPv4",
"isDefaultContainer": null,
"isXenon": false,
```

```

"powerShellVersion": null,
"preWarmedInstanceCount": null,
"publicNetworkAccess": null,
"publishingUsername": null,
"push": null,
"pythonVersion": null,
"remoteDebuggingEnabled": null,
"remoteDebuggingVersion": null,
"requestTracingEnabled": null,
"requestTracingExpirationTime": null,
"scmIpSecurityRestrictions": [
  {
    "action": "Allow",
    "description": "Allow all access",
    "headers": null,
    "ipAddress": "Any",
    "name": "Allow all",
    "priority": 2147483647,
    "subnetMask": null,
    "subnetTrafficTag": null,
    "tag": null,
    "vnetSubnetResourceId": null,
    "vnetTrafficTag": null
  }
],
"scmIpSecurityRestrictionsDefaultAction": null,
"scmIpSecurityRestrictionsUseMain": null,
"scmMinTlsVersion": null,
"scmType": null,
"tracingOptions": null,
"use32bitWorkerProcess": null,
"virtualApplications": null,
"vnetName": null,
"vnetPrivatePortsCount": null,
"vnetRouteAllEnabled": null,
"webSocketEnabled": null,
"websiteTimeZone": null,
"windowsFxVersion": null,
"xManagedServiceIdentityId": null
},
"sku": "Dynamic",
"slotSwapStatus": null,
"sshEnabled": null,
"state": "Running",
"storageAccountRequired": false,
"suspendedFill": null,
"systemData": null,
"tags": null,
"targetSwapSlot": null,
"trafficManagerHostNames": null,
"type": "Microsoft.Web/sites",
"usageState": "Normal",
"virtualNetworkSubnetId": null,
"workloadProfileName": null
}
vaibhavgupta@VF0905WQ05F-vaibhavgupta BlobArchiveFunctions %

```

Function App URL
(https://FUNC_NAME.azurewebsites.net)

func-lab-app-13938.azurewebsites.net

```

"defaultHostName": "func-lab-app-13938.azurewebsites.net",
"dnsConfiguration": {
  "dnsAltServer": null,
  "dnsLegacySortOrder": null,
  "dnsMaxCacheTimeout": null,
  "dnsRetryAttemptCount": null,
  "dnsRetryAttemptTimeout": null,
  "dnsServers": null
},
"enabled": true,
"enabledHostNames": [
  "func-lab-app-13938.azurewebsites.net",
  "func-lab-app-13938.scm.azurewebsites.net"
]

```

Step 2 — Assign a Managed Identity

```
az functionapp identity assign --name %FUNC_NAME% --resource-group %RG%
```

18. Capture the identity's principal ID:

```
for /f "delims=" %a in ('az functionapp identity show --name %FUNC_NAME% --resource-group %RG% --query principalId -o tsv') do set PRINCIPAL_ID=%a
```

```
vaibhavgupta@FVG986WQ85F-vaibhavgupta BlobArchiveFunctions % az functionapp config appsettings set \
--name $FUNC_NAME \
--resource-group $RG \
--settings "BlobStorageConnectionString=DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=func1abs13938;AccountKey=cnldSEat4bKH2lNgw4gkGby3or87c3AmVdc8DNGTzu50v8Y++U8PGSg1faE
fMt+trFTXLaR6sy7DnAStBx4qjQ==;BlobEndpoint=https://func1abs13938.blob.core.windows.net/FileEndpoint=https://func1abs13938.file.core.windows.net/QueueEndpoint=https://func1abs13938.queue.core.window
s.net/TableEndpoint=https://func1abs13938.table.core.windows.net/"
{
  "name": "FUNCTIONS_WORKER_RUNTIME",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITE_USE_PLACEHOLDER_DOTNETISOLATED",
  "slotSetting": false,
  "value": null
},
{
  "name": "FUNCTIONS_EXTENSION_VERSION",
  "slotSetting": false,
  "value": null
},
{
  "name": "AzureWebJobsStorage",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITE_CONTENTAZUREFILECONNECTIONSTRING",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITE_CONTENTSHARE",
  "slotSetting": false,
  "value": null
},
{
  "name": "AzureWebJobsDashboard",
  "slotSetting": false,
  "value": null
},
{
  "name": "BlobStorageConnectionString",
  "slotSetting": false,
  "value": null
}
}
```

Step 3 — Grant the Identity Key Vault Access

```
az keyvault set-policy ^
--name %KV_NAME% ^
--object-id %PRINCIPAL_ID% ^
--secret-permissions get list
```

Step 4 — Set the Key Vault Reference App Setting

Instead of pasting the connection string directly, use a Key Vault Reference. Azure resolves the `@Microsoft.KeyVault(...)` syntax automatically and injects the real value at runtime:

```
az functionapp config appsettings set ^
--name %FUNC_NAME% ^
--resource-group %RG% ^
--settings
"BlobStorageConnectionString=@Microsoft.KeyVault(VaultName=%KV_NAME%;SecretName=BlobStorageConnectionString"
```

Tip

This is Key Vault Reference syntax. Your function code reads `BlobStorageConnectionString` exactly as before — Azure quietly swaps in the real value from Key Vault before handing it to your code. No code changes required.


```
vaibhavgupta@FVG998WQ85F-vaibhavgupta BlobArchiveFunctions % az functionapp config appsettings set \
--name $FUNC_NAME \
--resource-group $RG \
--settings "BlobStorageConnectionString=DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=func1absa13938;AccountKey=cnldSEat4bKHZlNgw4gkGby3or87c3AmVdc8DN0GTzu5OvBY++U8PQ5glfaE
Fm+trFTXLaR6sy7DhAST8x4qjQ==;BlobEndpoint=https://func1absa13938.blob.core.windows.net;/FileEndpoint=https://func1absa13938.file.core.windows.net;/QueueEndpoint=https://func1absa13938.queue.core.window
s.net;/TableEndpoint=https://func1absa13938.table.core.windows.net/"
[
  {
    "name": "FUNCTIONS_WORKER_RUNTIME",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITE_USE_PLACEHOLDER_DOTNETISOLATED",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "FUNCTIONS_EXTENSION_VERSION",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "AzureWebJobsStorage",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITE_CONTENTAZUREFILECONNECTIONSTRING",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITE_CONTENTSHARE",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "AzureWebJobsDashboard",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "BlobStorageConnectionString",
    "slotSetting": false,
    "value": null
  }
]
```

Step 5 — Publish and Deploy

```
func azure functionapp publish %FUNC_NAME%
```

```
vaibhavgupta@FVG998WQ85F-vaibhavgupta BlobArchiveFunctions % func azure functionapp publish $FUNC_NAME
Detected target framework: net10.0
Determining projects to restore...
All projects are up-to-date for restore.
Determining projects to restore...
Restored /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/obj/release/net10.0/WorkerExtensions/WorkerExtensions.csproj (in 1.49 sec).
WorkerExtensions -> /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/obj/release/net10.0/WorkerExtensions/bin/Release/net10.0/Microsoft.Azure.Functions.Worker.E
xtensions.dll
BlobArchiveFunctions -> /Users/vaibhavgupta/Desktop/Intern Work/genspark/Day 49 July 2 Azure/BlobArchiveFunctions/bin/publish/BlobArchiveFunctions.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:07.20

Getting site publishing info...
[2024-07-03T07:13:55.476Z] Starting the function app deployment...
Creating archive for current directory...
Uploading 3.04 MB [#####]
Upload completed successfully.
Deployment completed successfully.
[2024-07-03T07:16:22.215Z] Syncing triggers...
Functions in func-lab-app-13938:
  ManualArchive - [httpTrigger]
    Invoke url: https://func-lab-app-13938.azurewebsites.net/api/manualarchive
  NightlyArchive - [timerTrigger]
```

Step 6 — Test the Deployed HTTP Trigger

19. Upload a test file to blob storage first:

```
az storage blob upload ^
--account-name %SA_NAME% ^
--container-name uploads ^
--name cloudtest.txt ^
--data "uploaded for cloud test" ^
--auth-mode login
```

20. Trigger the ManualArchive function in the cloud:

```
curl -X POST "https://%FUNC_NAME%.azurewebsites.net/api/ManualArchive?days=0"
```

Expected Result

Expected response: Archive complete. Files moved: 1

```
"client_request_id": "b26f0708-76af-11f1-9818-02de72963a13",
"content_md5": "f094da9f9f94PFS3ConFysw==",
"date": "2026-07-03T07:20:41:00:00",
"encryption_key_sha256": null,
"encryption_scope": null,
"etag": "\"0x80ED8D03973ED2AE\"",
"lastModified": "2026-07-03T07:20:42:00:00",
"request_id": "fa361bb3-e01e-0000-5bbc-0add40000000",
"request_server_encrypted": true,
"structured_body": null,
"version": "2026-04-06",
"version_id": null
}
vaibhavgupta@FVF090WQ05F-vaibhavgupta BlobArchiveFunctions % curl -X POST "https://func-lab-app-13938.azurewebsites.net/api/ManualArchive?days=0"
Archive complete. Files moved: 1
```

curl response from the deployed function — Archive complete. Files moved: 1

21. Verify the blob moved to archive:

```
az storage blob list --account-name %SA_NAME% --container-name archive --auth-mode login -
o table
```

```
vaibhavgupta@FVF090WQ05F-vaibhavgupta BlobArchiveFunctions % az storage blob list --account-name func1ab3a13938 --container-name archive --auth-mode key -o table
There are no credentials provided in your command and environment, we will query for account key for your storage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credentials.

You also can add '--auth-mode login' in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information about environment variable usage.
Name          Blob Type  Blob Tier  Length  Content Type  Last Modified  Snapshot
-----
cloudtest.txt  BlockBlob  Hot        23      application/octet-stream  2026-07-03T07:20:54+00:00
testfile.txt   BlockBlob  Hot        18      application/octet-stream  2026-07-03T07:03:49+00:00
vaibhavgupta@FVF090WQ05F-vaibhavgupta BlobArchiveFunctions %
```


Lab 9: Cleanup

Important

This permanently deletes all resources. Only run when you are completely finished with the lab.

```
az group delete --name %RG% --yes --no-wait
```

Screenshot

az group delete accepted — all resources queued for deletion

Reflection Questions

1. What is the difference between a Timer Trigger and an HTTP Trigger? When would you choose one over the other?

A Timer Trigger runs automatically on a set schedule, while an HTTP Trigger runs only when someone specifically calls its web URL. Choose a Timer for routine background tasks (like nightly cleanups) and an HTTP trigger for on-demand actions (like a user clicking a button).

2. What does the cron expression `0 0 0 * * *` mean? How would you change it to run every Monday at 6:00 AM?

The expression `0 0 0 * * *` means the function runs every day exactly at midnight. To change it to run every Monday at 6:00 AM, you would use the expression `0 0 6 * * 1`.

3. What is a Key Vault Reference? How is it different from storing the connection string directly as an app setting?

A Key Vault Reference is a pointer that tells the app to securely fetch a secret from a digital vault only when it needs it. This is much safer than saving the raw, readable password directly in the app's standard settings.

4. The `NightlyArchive` function runs at midnight UTC. If your business is in India (IST = UTC+5:30), what cron expression would run it at midnight IST?

Since IST is 5.5 hours ahead of UTC, midnight in India is actually 6:30 PM (18:30) the day before in UTC. The correct cron expression to run at midnight IST is 0 30 18 * * *.

Common Errors

Error	Fix
func: command not found	Azure Functions Core Tools not installed. Run: <code>npm install -g azure-functions-core-tools@4 --unsafe-perm true</code>
Variable not set (%SA_NAME% prints literally)	Session was closed. Re-run all set commands from Lab 1 using your saved SUFFIX value
BlobStorageConnectionString is not configured	Check local.settings.json — the value must be the actual connection string, not a placeholder
403 on Key Vault (deployed)	Managed Identity access policy not set or wrong principalId — re-run <code>az keyvault set-policy</code> with the correct PRINCIPAL_ID
Key Vault Reference shows as literal string	The Function App identity was not granted get permission on Key Vault secrets — check Step 3 of Lab 8
func start fails — port in use	Another process is using port 7071. Close it or run: <code>func start --port 7072</code>
dotnet build errors on the function files	Ensure the <code>func new</code> command ran successfully and both .cs files exist in the project folder
Blob not archived — count is 0	The file is newer than the days threshold. Test with <code>?days=0</code> to archive all files regardless of age